

# Circula

---

## Marketplace de Economia Circular do Campus

### Apostila Técnica de Estudo

**Francisco Magno Quezado** Desafio Técnico — Laboratório de Inovação Vortex (UNIFOR) Processo Seletivo para Estágio Full-Stack 2026

Aplicação: desafio-vortex-api.vercel.app  
github.com/Franmagno06/Desafio-Vortex

API: circula-api.onrender.com

Repositório:

# Como usar esta apostila

---

Este material consolida os conceitos técnicos das sete sprints de desenvolvimento do Circula, organizados para **estudo** e não para consulta rápida.

A estrutura é:

| Parte | Conteúdo                                             |
|-------|------------------------------------------------------|
| 1     | Visão geral e arquitetura — o mapa mental do projeto |
| 2     | Conceitos do backend                                 |
| 3     | Conceitos do frontend                                |
| 4     | PWA e Service Worker                                 |
| 5     | Infraestrutura e deploy                              |
| 6     | Os erros que ensinaram — os 15 bugs documentados     |
| 7     | <b>Banco de perguntas e respostas</b> para a banca   |
| 8     | Glossário                                            |

**Sugestão de uso:** leia as partes 1 a 5 uma vez para formar o mapa mental. Depois use a parte 7 como simulado — leia a pergunta, responda em voz alta, e só então confira. Se travar numa resposta, volte à parte correspondente.

A parte 6 é a mais valiosa para a entrevista: erro entendido demonstra mais domínio que acerto de primeira.

---

# Parte 1 — Visão geral e arquitetura

## 1.1 O problema

Todo fim de semestre o mesmo ciclo se repete no campus: veteranos com livros, jalecos e calculadoras parados numa gaveta; calouros gastando caro exatamente nesses materiais. O material existe — falta um canal confiável ligando os dois lados **dentro da universidade**.

O Circula é esse canal: uma aplicação única com duas caras — uma landing page rica no desktop, para descoberta, e um aplicativo instalável no celular, para anunciar rápido.

## 1.2 Stack escolhida

| Camada    | Tecnologia                                          | Motivo da escolha                                                                    |
|-----------|-----------------------------------------------------|--------------------------------------------------------------------------------------|
| Backend   | Node + TypeScript + <b>Express 5</b>                | Framework já dominado — o critério mais pesado do edital é explicar o próprio código |
| Validação | <b>Zod 4</b>                                        | Schemas viram tipos TypeScript e documentação OpenAPI, sem duplicação                |
| ORM       | <b>Prisma 6</b>                                     | Schema declarativo, migrations versionadas                                           |
| Banco     | <b>PostgreSQL</b> (Neon)                            | Bônus explícito do edital; mesmo banco em dev e produção                             |
| Auth      | <b>JWT + bcryptjs</b>                               | Autoria própria; <b>bcryptjs</b> é JS puro, sem compilação nativa                    |
| Frontend  | <b>React 19 + Vite 8</b>                            | Controle total do Service Worker, que o edital pede para explicar                    |
| Estilo    | <b>Tailwind CSS 4</b>                               | Tema em CSS ( <b>@theme</b> ), responsividade rápida                                 |
| Dados     | <b>TanStack Query 5</b>                             | Cache, estados de carregamento e base do offline                                     |
| PWA       | <b>vite-plugin-pwa</b><br>( <b>injectManifest</b> ) | Service Worker escrito à mão                                                         |

## 1.3 O monorepo

```
circula/  
├─ apps/  
│   ├─ api/           API REST (Express 5)  
│   └─ web/           PWA (React 19 + Vite)  
└─ packages/  
    └─ shared/        enums, schemas Zod e regras de negócio
```

**O pacote compartilhado é a espinha do projeto.** Ele define o vocabulário do domínio uma vez só e entrega para os dois lados. Se uma categoria nova nascer lá, a API passa a aceitá-la e o PWA passa a exibi-la — sem duplicação.

Os workspaces do npm criam um **link simbólico** de `node_modules/@circula/shared` para `packages/shared`. É isso que faz `import { CATEGORIES } from '@circula/shared'` funcionar nos dois apps sem publicar nada no npm.

## 1.4 O fluxo de uma requisição

POST /api/v1/announcements

```
|
|→ helmet          cabeçalhos de segurança
|→ cors            allowlist de origens
|→ express.json    parse do corpo (limite 100kb)
|→ pino-http       log estruturado
|→ rateLimit       100 req / 15 min por IP
|
|→ requireAuth     valida o JWT, preenche req.userId
|→ validateBody    Zod: valida E transforma
|
|→ rota            extrai dados, chama o service
|→ service         regra de negócio (não conhece HTTP)
|→ repository      interface (Prisma em produção, memória nos testes)
|→ mapper          registro do banco → DTO público
|
|→ res.status(201).json()
```

↓ em caso de erro, em qualquer ponto

```
errorHandler → { "error": { "code", "message", "details" } }
```

**A ordem dos middlewares é a ordem de execução, e ela é funcional, não estética:** `trust proxy` precisa valer antes do rate limit, senão todos os usuários compartilhariam o IP do proxy; o CORS barra origem não autorizada antes de gastar processamento desserializando o corpo.

## Parte 2 — Conceitos do backend

### 2.1 Camadas e injeção de dependência

Cada recurso tem quatro arquivos:

| Arquivo                 | Responsabilidade             | O que ele <b>não</b> sabe |
|-------------------------|------------------------------|---------------------------|
| <code>routes</code>     | método, caminho, status HTTP | regra de negócio          |
| <code>service</code>    | regra de negócio             | que existe HTTP           |
| <code>repository</code> | acesso a dados               | regra de negócio          |
| <code>mapper</code>     | registro do banco → DTO      | qualquer uma das outras   |

O service **não importa o Prisma**. Ele recebe um objeto que cumpre a interface `AnnouncementsRepository` :

```
export function createAnnouncementsService(repository: AnnouncementsRepository) { ... }
```

Em produção entra a implementação Prisma; nos testes, uma em memória. Isso se chama **injeção de dependência**.

**A motivação foi concreta, não teórica:** o workflow do GitHub Actions não sobe um PostgreSQL. Sem essa separação, ou o CI ficaria sem testes, ou seria preciso manter um banco no pipeline só para exercitar regras que não dependem de banco nenhum. Resultado: **67 testes em cerca de 8 segundos**, sem serviço externo.

### 2.2 A regra de negócio central

**Doação e troca não têm preço.** Ela vive em `packages/shared/src/domain/rules.ts` como uma função pura:

```
export function checkPriceAgainstType(
  type: ItemType,
  priceCents: number | null,
): PriceRuleViolation | null {
  if (type === 'VENDA') {
    if (priceCents === null) return 'PRICE_REQUIRED_FOR_SALE';
    if (priceCents < MIN_PRICE_CENTS) return 'PRICE_BELOW_MINIMUM';
    return null;
  }
  return priceCents === null ? null : 'PRICE_NOT_ALLOWED';
}
```

"Pura" significa que ela só recebe dados e devolve um resultado — não conhece HTTP, banco nem React. Por isso pode valer nos **três** lugares onde a regra precisa existir:

1. no schema Zod (validação da requisição na API);
2. no service (quando um PATCH parcial altera o tipo sem reenviar o preço);
3. no formulário do PWA (o campo de preço **deixa de existir** quando é doação).

Uma regra, um arquivo, três usos. É isso que impede o clássico "o front deixou passar mas a API recusou".

## 2.3 Dinheiro em centavos

`priceCents` é sempre um inteiro. R\$ 19,90 vira `1990`.

```
0.1 + 0.2; // 0.30000000000000004
```

Ponto flutuante não representa decimais exatamente. Guardando centavos, a classe inteira de bugs de arredondamento deixa de existir. A conversão para `"R$ 19,90"` acontece só na exibição.

## 2.4 Exclusão lógica (soft delete)

`DELETE` não apaga a linha — preenche `deletedAt`:

```
await prisma.announcement.update({ where: { id }, data: { deletedAt: new Date() } });
```

**Vantagens:** histórico preservado, exclusão reversível, estatísticas históricas intactas.

**O preço:** uma disciplina que não pode falhar — toda consulta precisa filtrar `deletedAt: null`. Se um único `findMany` esquecer, anúncios excluídos voltam à vitrine. Por isso o filtro é a **primeira linha** do `buildWhere()`.

## 2.5 Índices

```
@@index([deletedAt, status, createdAt]) // listagem pública ordenada
@@index([deletedAt, category])          // filtro por categoria
@@index([authorId, deletedAt])          // "meus anúncios"
```

Índice não é enfeite — é a diferença entre o banco ler três linhas ou varrer a tabela inteira. Cada um foi desenhado a partir de uma consulta que a aplicação **realmente** faz.

**A ordem das colunas importa:** o Postgres usa um índice composto da esquerda para a direita.

## 2.6 Fail fast nas variáveis de ambiente

```
const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error(JSON.stringify(z.treeifyError(parsed.error), null, 2));
  process.exit(1);
}
```

Sem essa checagem, um `DATABASE_URL` ausente só explodiria na primeira requisição que tocasse o banco — provavelmente em produção. Com ela, o processo morre no boot dizendo exatamente qual variável está errada.

## 2.7 Erros desacoplados do HTTP

Services lançam `AppError` com um **código semântico**:

```
throw notFound('Anúncio não encontrado.');// o service não sabe o que é "404"
```

Um único middleware traduz código → status HTTP e monta o envelope. Toda resposta de erro tem o mesmo formato — inclusive rotas inexistentes, que sem o `notFoundHandler` devolveriam o HTML `Cannot GET /x` do Express, violando o requisito de "JSON estritamente".

## 2.8 Autenticação

### O que é um JWT

Três partes separadas por ponto: `header.payload.signature`. As duas primeiras são **base64, não criptografia** — qualquer pessoa lê o conteúdo.

**JWT garante integridade, não sigilo.** A assinatura prova que o payload não foi alterado depois de emitido. Por isso o payload aqui carrega apenas o id do usuário, que já é público.

## bcrypt: a lentidão é o recurso

```
const BCRYPT_ROUNDS = 12; // ~250ms por hash
```

Todo instinto diz para tornar o código rápido. Aqui é o contrário: **o custo é a defesa**. Quem roubar o banco precisa gastar 250ms por tentativa em cada senha. Com MD5 ou SHA-256, o mesmo ataque testaria bilhões por segundo.

Duas propriedades: **salt automático** (duas senhas iguais geram hashes diferentes, inutilizando rainbow tables) e **limite de 72 bytes** (o bcrypt trunca em silêncio acima disso — daí o `.max(72)` no schema).

## Timing attack

```
const DUMMY_HASH = bcrypt.hashSync('senha-que-nunca-sera-usada', BCRYPT_ROUNDS);
const matches = await bcrypt.compare(input.password, user?.passwordHash ?? DUMMY_HASH);
```

Por que comparar contra um hash falso quando o usuário nem existe?

| Cenário           | Sem o dummy                 | Tempo  |
|-------------------|-----------------------------|--------|
| E-mail não existe | retorna 401 na hora         | ~5ms   |
| E-mail existe     | roda o bcrypt e retorna 401 | ~250ms |

Essa diferença é **mensurável de fora**. Um atacante cronometra as respostas e descobre quais e-mails têm conta, sem acertar uma senha. Chama-se **ataque de canal lateral**.

## Mensagens deliberadamente pouco informativas

Login responde a mesma mensagem para "e-mail não existe" e "senha errada". Dizer qual dos dois falhou transformaria a tela num validador de e-mails cadastrados.

**Contraste proposital:** o **cadastro** devolve 409 dizendo que o e-mail já existe. Ali a informação é necessária — a pessoa precisa saber que deve fazer login. A regra não é "nunca revele nada", é revelar só onde há motivo legítimo.

---



## Parte 3 — Conceitos do frontend

### 3.1 Por que TanStack Query

O padrão de tutorial ( `useState` + `useEffect` + `fetch` ) tem quatro problemas:

1. **Sem cache** — voltar a uma tela refaz a requisição.
2. **Race condition** — trocar filtro rápido dispara duas requisições; se a primeira responder por último, a tela mostra o resultado errado.
3. **Duplicação** — dois componentes que precisam do mesmo dado fazem duas requisições.
4. **Boilerplate** — três `useState` repetidos por tela.

A chave de cache inclui os filtros, então trocar de categoria vira outra query com cache próprio — voltar mostra o resultado instantâneo.

### 3.2 Os quatro estados de uma tela

A maior parte das telas mal feitas trata só "carregando" e "pronto". A vitrine trata quatro:

| Estado     | O que aparece                         |
|------------|---------------------------------------|
| Carregando | Skeletons com a forma exata dos cards |
| Erro       | Mensagem + botão "Tentar de novo"     |
| Vazio      | Explica e oferece uma saída           |
| Com dados  | A vitrine                             |

O **vazio** é o mais esquecido e o mais visível numa demonstração.

E existe um quinto, invisível, que causou o bug mais difícil do projeto — ver Parte 6.

### 3.3 `isPending` × `isFetching`

| Estado                  | Significado                 | O que a UI faz  |
|-------------------------|-----------------------------|-----------------|
| <code>isPending</code>  | Nunca houve dado            | skeletons       |
| <code>isFetching</code> | Já existe dado, revalidando | esmaece o atual |

Com `placeholderData: (previous) => previous`, trocar de filtro mantém a lista anterior visível em vez de piscar skeletons.

### 3.4 Sessão é Context, não Query

O TanStack Query cuida de **dados do servidor que podem ser revalidados**. O token é outra coisa: é o estado local que determina **quem** faz as requisições. Numa query, criaria a dependência circular de precisar do token para buscar o token.

### 3.5 O guard de rota tem três estados

```
if (isLoading) return <Spinner />; // ← esquecer isto é o bug
if (!isAuthenticated) return <Navigate to="/entrar" />;
return children;
```

No primeiro carregamento existem **três** estados: validando o token guardado, autenticado, não autenticado. Tratando só os dois últimos, todo F5 numa rota protegida manda para o login — porque no instante da primeira renderização o `user` ainda é `null`, mesmo com token válido no `localStorage`.

### 3.6 Invalidação de cache é manual

Depois de criar um anúncio, a vitrine, "meus anúncios", as estatísticas e a contagem dos chips ficam desatualizados. O React Query **não adivinha** essa dependência:

```
void queryClient.invalidateQueries({ queryKey: ['announcements'] }); // pela raiz
void queryClient.invalidateQueries({ queryKey: ['stats'] });
void queryClient.invalidateQueries({ queryKey: ['categories'] });
```

Invaldar pela **raiz** marca todas as listagens de uma vez, com qualquer combinação de filtros — é por isso que as chaves são hierárquicas.

### 3.7 Debounce evita race condition

```
const debouncedSearch = useDebounceValue(search, 400);
```

Digitar "arduino" sem debounce dispara **sete** requisições. Pior que o desperdício: a resposta da terceira pode chegar depois da sétima e sobrescrever o resultado correto.

O mecanismo é o `clearTimeout` no cleanup do `useEffect`: cada tecla cancela o timer anterior.

### 3.8 Acessibilidade por construção

| Escolha                                                                      | Por quê                                            |
|------------------------------------------------------------------------------|----------------------------------------------------|
| Chips são <code>&lt;button&gt;</code> , não <code>&lt;div onClick&gt;</code> | Foco por teclado e leitor de tela vêm de graça     |
| <code>aria-pressed</code> no filtro ativo                                    | A cor sozinha só comunica para quem enxerga        |
| <code>aria-live="polite"</code> na vitrine                                   | Trocar o filtro anuncia o novo resultado           |
| Card inteiro é <b>um</b> <code>&lt;a&gt;</code>                              | Dois links exigiriam dois Tab para o mesmo destino |
| <code>alt=""</code> em imagem decorativa                                     | O título ao lado já descreve o item                |
| <code>aria-describedby</code> ligando erro ao campo                          | O leitor anuncia <b>qual</b> é o problema          |

Nada disso foi acrescentado depois com auditoria — são decisões tomadas ao escrever cada componente.

### 3.9 Tailwind: o conflito de utilitárias

```
<Button variant="ghost" className="text-brand-100"> {/* ❌ */}
```

O botão fica com `text-ink-700` (da variante) e `text-brand-100` ao mesmo tempo. No Tailwind, duas classes que definem a mesma propriedade **não** se resolvem pela ordem no atributo `class` — vence a que aparece por último no **CSS gerado**.

A solução não é `!important` nem `tailwind-merge` : é **uma variante por contexto**.

## Parte 4 — PWA e Service Worker

### 4.1 O que é um Service Worker

Um script que roda **numa thread separada** e age como **proxy entre o app e a rede**. Toda requisição passa por ele antes de sair, e ele decide se responde do cache, vai à rede, ou os dois.

Três detalhes:

- **Ciclo de vida:** `install` → `activate` → `fetch`. Um SW novo fica "esperando" enquanto o antigo controla abas abertas.
- **Escopo global diferente:** não existe `window` nem `document`; o global é `self`.
- **Só funciona em HTTPS** (ou `localhost`) — um proxy sobre todo o tráfego seria uma arma em conexão insegura.

### 4.2 As seis estratégias

| O quê                        | Estratégia                    | Por quê                                                    |
|------------------------------|-------------------------------|------------------------------------------------------------|
| App shell                    | <b>Precache</b>               | É o que faz o app <b>abrir</b> offline                     |
| Rotas da SPA                 | <b>NavigationRoute</b>        | Qualquer rota é servida pelo mesmo <code>index.html</code> |
| Listagens da API             | <b>Stale-While-Revalidate</b> | Resposta instantânea + atualiza em background              |
| <code>/auth/*</code>         | <b>Network-First</b>          | Cache manteria sessão expirada como válida                 |
| Imagens                      | <b>Cache-First</b>            | A mesma URL sempre devolve a mesma imagem                  |
| <code>POST</code> de anúncio | <b>Background Sync</b>        | Reenvia sozinho quando a conexão voltar                    |

#### Stale-While-Revalidate na prática

```
1ª visita → rede (nada em cache ainda)
2ª visita → cache instantâneo + atualização em background
sem rede → cache, e a vitrine continua navegável
```

#### Background Sync

Se a pessoa toca em "Publicar" sem rede, a requisição vai para uma fila no **IndexedDB** e o navegador a reenvia sozinho quando a conexão voltar — **mesmo que o app já tenha sido fechado**. É o que separa "site que quebra offline" de "aplicativo".

## 4.3 Não guardar erro no cache

```
new CacheableResponsePlugin({ statuses: [0, 200] });
```

Sem isso, uma resposta 500 entraria no cache e seria servida como válida, possivelmente por dias. O `0` cobre respostas opacas (recursos de outra origem).

## 4.4 Duas camadas de cache

| Camada                   | Guarda                         |
|--------------------------|--------------------------------|
| Service Worker           | as <b>respostas HTTP</b>       |
| Persistir do React Query | o <b>estado do React Query</b> |

- **Só o SW:** ao reabrir offline, o React Query começa vazio e as queries são atendidas pelo cache do SW. Funciona, mas pisca skeletons antes.
- **Com o persistir:** os dados aparecem na primeira renderização, sem carregamento.

Detalhe: o `gcTime` precisou subir de 5 minutos para 24 horas. Com `gcTime` curto, o React Query descarta a query da memória antes de o app reabrir — e não sobra o que persistir.

## 4.5 Ícone `maskable`

O Android **recorta** o ícone na forma do sistema (círculo, squircle). Se a arte ocupar as bordas, o recorte come parte dela.

| Propósito             | Arte ocupa    | Onde aparece              |
|-----------------------|---------------|---------------------------|
| <code>any</code>      | 62% do canvas | aba do navegador, desktop |
| <code>maskable</code> | 44% do canvas | tela inicial do Android   |

O iOS ignora o manifesto para isso e exige `<link rel="apple-touch-icon">`. Sem essa tag, o iPhone usa **um print da página** como ícone.

## 4.6 O prompt de instalação

```
window.addEventListener('beforeinstallprompt', (event) => {
  event.preventDefault(); // impede o banner padrão do Chrome
  setInstallEvent(event); // guarda para o nosso botão
});
```

O evento só pode ser usado **uma vez**. E o **Safari não implementa** — no iPhone a instalação é manual, então o app detecta iOS e mostra as instruções.

---

## Parte 5 — Infraestrutura e deploy

### 5.1 Infraestrutura como código

`render.yaml` e `vercel.json` descrevem os serviços **em código**. Duas vantagens: a configuração fica versionada junto do código, e recriar o serviço é apertar um botão.

```
- key: DATABASE_URL
  sync: false # a Render PEDE o valor e o guarda cifrado
```

`sync: false` significa "esta variável existe, mas o valor não está aqui" — o arquivo é público.

### 5.2 Migrations antes do tráfego

```
"start:prod": "prisma migrate deploy && node dist/server.js"
```

A ordem evita uma janela em que o **código é novo e o schema é velho**. E o `&&` garante que, se a migration falhar, o servidor **não sobe** — melhor falhar visivelmente do que subir quebrado.

`migrate deploy` (produção) ≠ `migrate dev`. O segundo pode recriar o banco.

### 5.3 O cache que congela a aplicação

```
{ "source": "/sw.js", "headers": [{ "key": "Cache-Control", "value": "max-age=0" }] }
```

Se o Service Worker tivesse cache longo:

1. o navegador guardaria o `sw.js` antigo;
2. esse SW serve o app antigo, que está no **precache dele**;
3. a aplicação fica **congelada numa versão velha** e nenhum deploy chega ao usuário.

Já os arquivos em `/assets/` podem ter cache eterno porque têm **hash no nome**.

### 5.4 Variáveis `VITE_*`

São **embutidas no bundle em tempo de build** e vão para o JavaScript que qualquer pessoa lê. Mudar no painel sem redeploy não tem efeito, e segredo nenhum pode ir nelas.

## 5.5 CORS por ambiente

| Origem                   | Desenvolvimento | Produção |
|--------------------------|-----------------|----------|
| localhost:5173           | ✓               | ✗ 403    |
| 127.0.0.1:5173           | ✓               | ✗ 403    |
| 192.168.x.x (rede local) | ✓               | ✗ 403    |
| Domínio da Vercel        | ✓               | ✓        |
| Qualquer outro           | ✗               | ✗        |

A liberação local existe para testar o PWA no celular pelo IP da rede. Em produção vale só a allowlist explícita — verificado rodando a API compilada com `NODE_ENV=production`.



## Parte 6 — Os erros que ensinaram

Foram **15 erros documentados** ao longo do projeto. Estes são os que mais ensinam.

### 6.1 O bug que apagaria preços em produção

**Sintoma:** um `PATCH { "status": "RESERVADO" }` apagava o preço do anúncio.

**Causa:** o schema de criação tinha `.default(null)` no preço, e o de atualização derivava dele com `.partial()`.

```
PATCH { "status": "RESERVADO" }  
  ↓ parse  
{ status: 'RESERVADO', priceCents: null } ← o default foi aplicado!  
  ↓  
UPDATE announcements SET priceCents = NULL
```

`.partial()` torna o campo opcional, mas um `.default()` continua sendo aplicado quando o campo está ausente.

**Como foi pego:** rodando um script que exercitava cada regra e imprimia o resultado, **antes** de construir o resto em cima dos schemas.

### 6.2 Código inalcançável que parecia correto

O service tinha um bloco explícito zerando o preço ao converter venda em doação. O bloco **nunca executava**: três linhas acima, a validação já lançava 422 comparando o tipo novo com o preço antigo.

A intenção estava certa, a **ordem** estava errada — e o código *lia* como se funcionasse. Só um teste automatizado revelou.

Código morto não avisa que é morto.

### 6.3 O quarto estado de uma query

**Sintoma:** com a API fora do ar, a tela de Explorar ficava presa em skeletons **para sempre**. Sem erro, sem mensagem. E o botão "Tentar de novo" não fazia nada.

**Causa:** o TanStack Query tem quatro estados, não três:

|                    |                           |
|--------------------|---------------------------|
| pending + fetching | → carregando de verdade   |
| pending + paused   | → PAUSADO, esperando rede |
| error              | → falhou                  |
| success            | → tem dado                |

Quando a primeira requisição falha por erro de rede, ele **pausa** em vez de reportar erro. Nesse estado não erra, não tenta de novo e **ignora** `refetch()`.

#### A correção teve três partes:

1. `networkMode: 'always'` — quem responde offline aqui é o Service Worker, então pausar por achar que está sem rede atrapalha.
2. Tratar o estado `paused` na interface, com mensagem e botão.
3. `resetQueries` no botão, não `refetch()` — a query pausada ignora o refetch.

**O erro de método:** foram feitas duas tentativas de correção **por dedução**, ambas falharam. A terceira, depois de instrumentar o estado real com um build de depuração, acertou de primeira. **Meça antes de corrigir.**

## 6.4 O contraste invisível

Depois da troca de paleta, o botão "Explorar a vitrine" ficou com contraste de **1,02:1** — texto cinza-escuro sobre fundo azul.

Passou por lint, typecheck, 56 testes e build. **Só olhar a tela pegou.**

A causa foi o conflito de utilitárias do Tailwind (seção 3.9). A correção foi criar a variante `ghostOnDark`, elevando o contraste para **8,41:1** (AAA).

## 6.5 "Funciona na minha máquina" — o CI vermelho

O CI ficou vermelho por duas sprints sem eu perceber. Duas causas:

- **typecheck**: o workflow não rodava `prisma generate`. O cliente tipado do Prisma é um artefato **gerado**, não versionado; localmente existia, mas o runner começa do zero.
- **format:check**: o formatador do editor reescrevia arquivos **depois** do meu check local e antes do commit.

**A lição não é técnica:** CI verde existe para pegar exatamente o que passa por acidente no ambiente local. Ignorá-lo por duas sprints anulou o valor de tê-lo.

## 6.6 Cinco vezes em que a ferramenta de diagnóstico mentiu

| Sprint | O que a ferramenta disse                           | A verdade                                                                         |
|--------|----------------------------------------------------|-----------------------------------------------------------------------------------|
| 0      | <code>/health</code> respondeu <code>200 OK</code> | Era <b>outro projeto</b> na mesma porta                                           |
| 1      | O código parecia funcionar                         | O bloco era <b>inalcançável</b>                                                   |
| 2      | Erro <code>"Bearer ."</code> truncado              | O PowerShell <b>removeu</b> o <code>&lt;token&gt;</code> achando que era tag HTML |
| 3      | A vitrine estava vazia                             | Havia 8 cards; a leitura ocorreu <b>antes da hidratação</b> do React              |
| 5      | O manifesto estava corrompido                      | O console do Windows não <b>renderiza</b> o travessão; o arquivo estava certo     |

**200 OK prova que alguém respondeu, não que foi o servidor certo.** Confirmar por um segundo caminho virou regra de trabalho.

## Parte 7 — Banco de perguntas e respostas

---

### Arquitetura e decisões

#### P. Por que Express e não Fastify ou NestJS?

R.

Por experiência prévia. O critério mais pesado do edital é "*capacidade de explicar o próprio código com propriedade*" — um framework que eu já domino me deixa defender cada middleware; um framework novo criaria dependência da IA até para justificar as escolhas.

Além disso, o Express 5 traz propagação automática de erros em handlers `async`, o que elimina o `try/catch` repetido em toda rota.

#### P. Por que monorepo?

R.

O edital pede "uma aplicação única integrando uma API RESTful e uma interface responsiva instalável". O monorepo dá separação clara sem fragmentar a entrega em dois repositórios, e permite o pacote `@circula/shared`, que é o que garante que a mesma regra vale nos dois lados.

Usei npm workspaces em vez de Turborepo ou Nx porque estes trazem cache e orquestração que este projeto não precisa, ao custo de mais configuração para explicar.

#### P. Como o front e o back compartilham as mesmas regras?

R.

Pelo pacote `@circula/shared`, linkado via workspaces do npm. Enums de domínio, schemas Zod e as funções de regra de negócio ficam lá.

Existe até uma rota, `/health/contract`, que devolve os enums em tempo de execução — serve para provar que o link simbólico está funcionando.

### P. Onde fica a regra de negócio?

R.

Num arquivo só: `packages/shared/src/domain/rules.ts`. A função `checkPriceAgainstType` é pura — não conhece HTTP, banco nem React.

Ela é usada em três lugares: no schema Zod que valida a requisição, no service (para o PATCH parcial, que precisa mesclar com o registro atual) e no formulário do PWA, onde decide se o campo de preço existe.

## Banco de dados

### P. Por que o preço é um inteiro?

R.

Porque `0.1 + 0.2 !== 0.3` em ponto flutuante. Guardando centavos como inteiro ( `1990` em vez de `19.90` ), a classe inteira de bugs de arredondamento deixa de existir. A formatação para `R$ 19,90` acontece só na exibição.

### P. O DELETE apaga o registro?

R.

Não. É exclusão lógica: preenche a coluna `deletedAt`. Preserva histórico e permite reverter.

O custo é que **toda** consulta precisa filtrar `deletedAt: null` — por isso esse filtro é a primeira linha do `buildWhere()`, e existe um teste garantindo que um anúncio excluído some da listagem mas continue na tabela.

### P. Por que esses três índices?

R.

Cada um corresponde a uma consulta que a aplicação realmente faz: a listagem pública ordenada por data, o filtro por categoria e a tela "meus anúncios".

A ordem das colunas importa porque o Postgres usa índice composto da esquerda para a direita — `[deletedAt, status, createdAt]` serve exatamente à consulta `WHERE deletedAt IS NULL AND status = 'ATIVO' ORDER BY createdAt DESC`.

### P. Por que PostgreSQL e não SQLite?

R.

O edital aceita SQLite, mas lista banco relacional real em nuvem como diferencial. Mais importante: desenvolver em SQLite e publicar em Postgres cria divergência de comportamento (tipos, enums nativos, case-sensitivity) que só aparece no deploy. Usar o mesmo banco nos dois ambientes elimina a classe "funciona na minha máquina".

## Testes

### P. Como você testa sem um banco no CI?

R.

O service depende de uma **interface** ( `AnnouncementsRepository` ), não do Prisma. Em produção entra a implementação Prisma; nos testes, uma implementação em memória que respeita o mesmo contrato.

São 67 testes em cerca de 8 segundos, sem nenhum serviço externo no pipeline.

### P. Que tipo de teste você escreveu?

R.

Testes de **integração**, não unitários. A requisição atravessa a pilha inteira — helmet, CORS, body parser, autenticação JWT, validação Zod, rota, service, mapper — e só a persistência é substituída.

O `createApp()` devolve a aplicação sem chamar `listen()`, então o Supertest sobe um servidor efêmero por teste, sem ocupar porta fixa.

## Autenticação e segurança

### P. Onde a senha fica guardada?

R.

Só o hash bcrypt, na coluna `passwordHash`. A senha em texto puro nunca é persistida nem registrada em log. Há um teste verificando que o valor armazenado começa com `$2` (prefixo do bcrypt) e é diferente da senha enviada.

### P. Por que 12 rodadas de bcrypt?

R.

Porque a lentidão é a defesa: cerca de 250ms por hash. Quem roubar o banco precisa gastar isso por tentativa em cada senha. Com um hash rápido, o mesmo ataque testaria bilhões por segundo. 12 é o equilíbrio entre custo para o atacante e latência aceitável no login.

### P. Alguém pode ler o conteúdo do token?

R.

Sim, e isso é esperado — as duas primeiras partes de um JWT são base64, não criptografia. O JWT garante **integridade**, não sigilo: a assinatura prova que o payload não foi alterado. Por isso ele carrega apenas o id do usuário, que já é público.

### P. Por que o login não diz se o e-mail existe?

R.

Porque isso transformaria a tela de login num validador de quais e-mails têm conta. Por isso também existe o `DUMMY_HASH`: sem ele, a **diferença de tempo** entre os dois casos (5ms contra 250ms) permitiria a mesma enumeração pelo relógio, mesmo com mensagem idêntica.

### P. Onde o token fica no cliente, e por quê?

R.

No `localStorage`, porque o PWA e a API vivem em domínios diferentes (Vercel e Render) — cookie cross-site exigiria `SameSite=None; Secure`, CORS com credenciais e sessão no servidor.

A troca é consciente: ficamos vulneráveis a XSS, mas imunes a CSRF por construção, porque o navegador nunca anexa esse cabeçalho sozinho. Está registrado no ADR 010.

## Frontend

### P. Por que TanStack Query e não fetch com useEffect?

R.

Por cache, deduplicação de requisições, proteção contra race condition ao trocar filtros e eliminação do boilerplate de três `useState` por tela. E, na Sprint 5, ele virou a base do funcionamento offline, persistindo o cache no navegador.

**P. As estatísticas são reais?**

**R.**

Sim. O edital pedia simuladas, mas `GET /api/v1/stats` executa `COUNT` no PostgreSQL. Dá para provar ao vivo comparando os números da tela com a resposta da API.

**P. O usuário consegue criar uma doação com preço?**

**R.**

Pela interface, não: o campo não existe quando o tipo é doação — o estado inválido é **inalcançável**. E mesmo montando a requisição na mão, a API recusa com 422, porque a mesma função de regra roda nos dois lados.

**P. Como a lista atualiza depois de criar um anúncio?**

**R.**

Por invalidação de cache. Invalido a raiz `['announcements']`, o que marca todas as listagens como obsoletas de uma vez, mais `/stats` e `/categories`, que dependem da contagem. O React Query refaz só as que estão na tela.

**P. Como você garante acessibilidade?**

**R.**

Por escolhas de estrutura, não por auditoria depois: elementos semânticos (`button`, `a`, `dl`), `aria-pressed` nos filtros, `aria-live` na vitrine, `aria-describedby` ligando erro ao campo, foco visível e respeito a `prefers-reduced-motion`.



## PWA

### P. Explique o Service Worker do seu projeto.

R.

É um proxy entre o app e a rede, rodando em thread separada. Escrevi seis regras:

- **precache** do app shell — é o que faz o app abrir offline;
- **navegação** devolve o `index.html` para qualquer rota da SPA;
- **listagens** usam stale-while-revalidate — cache instantâneo + atualização em background;
- `/auth` usa network-first, porque cache manteria sessão expirada;
- **imagens** usam cache-first, porque a URL nunca muda de conteúdo;
- **POST de anúncio** usa Background Sync, que reenfileira a publicação feita offline e reenvia sozinho quando a conexão volta.

### P. Por que escreveu o SW à mão em vez de gerar?

R.

Porque o `generateSW` produziria uma caixa-preta, e o edital pede que eu explique a lógica do Service Worker. Usei `injectManifest`: escrevo o SW e o plugin só injeta a lista de arquivos do precache.

### P. Por que dois arquivos de ícone diferentes?

R.

Porque o Android recorta o ícone na forma do sistema. O `maskable` tem a arte menor, dentro da zona segura, para não perder as bordas no recorte. Usar a mesma imagem faz o ícone aparecer cortado no celular.

### P. O que acontece se eu criar um anúncio sem internet?

R.

O Background Sync coloca a requisição numa fila no IndexedDB e o navegador a reenvia quando a conexão voltar, mesmo se o app tiver sido fechado. Ao sincronizar, o SW avisa as abas abertas para atualizarem a lista.

## Deploy

### P. Como funciona o seu deploy?

R.

A configuração está versionada: `render.yaml` descreve o serviço da API e `vercel.json` o do PWA. Publicar é conectar o repositório e preencher os segredos, marcados como `sync: false` para não entrarem no arquivo público.

### P. E as migrations de banco?

R.

Rodam no `start:prod`, com `prisma migrate deploy` **antes** de o servidor subir. Se falharem, o servidor não sobe. E uso `migrate deploy`, não `migrate dev` — o segundo pode recriar o banco.

### P. O que muda entre desenvolvimento e produção?

R.

O CORS deixa de aceitar localhost e a rede local; os logs saem em JSON em vez de coloridos; o HSTS é ativado; e o stack trace para de aparecer nas respostas de erro. Verifiquei os quatro rodando a API compilada com `NODE_ENV=production`.

### P. Por que o Service Worker não pode ser cacheado?

R.

Porque o navegador guardaria o SW antigo, que serve o app antigo do precache dele. A aplicação ficaria congelada numa versão velha e nenhum deploy chegaria ao usuário.

## Uso de IA

### P. Como você usou IA neste projeto?

R.

Como ferramenta, ao longo das sete sprints, com tudo registrado no Diário de Bordo. Três decisões definiram o uso:

1. **Não comecei pedindo código.** O primeiro prompt pedia um plano — stacks, arquitetura e sprints — para eu autorizar antes de qualquer linha ser escrita.
2. **Recusei recomendações.** A IA sugeriu Fastify; escolhi Express porque preciso explicar o código na banca.
3. **Exigi registro dos erros.** Sem isso, os bugs do caminho seriam corrigidos em silêncio e não virariam aprendizado.

### P. Qual foi o erro mais grave que a IA cometeu?

R.

Um schema em que o `.default(null)` sobrevivia ao `.partial()`. Na prática, um usuário que apenas reservasse o próprio anúncio teria **o preço apagado do banco**.

Peguei porque, antes de construir em cima dos schemas, rodei um script exercitando cada regra e vi `priceCents: null` numa saída onde não deveria estar. Hoje existe um teste de regressão fixando o comportamento.

### P. Qual foi o bug mais difícil de resolver?

R.

Uma tela presa em skeletons com a API fora do ar. Descobri que o TanStack Query tem um quarto estado — `pending` + `fetchStatus: 'paused'` — em que não erra, não tenta de novo e ignora `refetch()`.

O mais instrutivo foi o **método**: houve duas tentativas de correção por dedução, ambas falharam. Só depois de instrumentar o estado real a correção saiu de primeira. Ficou a regra de **medir antes de corrigir**.

**P. A IA escreveu tudo? Você entende o código?**

**R.**

A IA escreveu a maior parte do código, e isso está declarado no Diário de Bordo — omitir seria desonesto. O que fiz foi **curadoria**: defini a arquitetura antes de qualquer linha, recusei recomendações que me deixariam sem conseguir defender o resultado, exigi que cada decisão viesse com justificativa no comentário, e verifiquei o comportamento em vez de aceitar o código pela aparência.

Os 15 erros documentados são a prova de que a verificação aconteceu: cinco deles a IA cometeu e eu peguei testando, e em cinco ocasiões a própria ferramenta de diagnóstico mentiu — só descobri porque conferi por um segundo caminho.

---

## Parte 8 — Glossário

| Termo                         | Significado                                                                                                         |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>ADR</b>                    | <i>Architecture Decision Record</i> — registro de uma decisão, com contexto e consequência                          |
| <b>Background Sync</b>        | API que reenfileira requisições feitas offline e as reenvia quando a rede volta                                     |
| <b>bcrypt</b>                 | Algoritmo de hash de senha deliberadamente lento, com salt automático                                               |
| <b>Cache-First</b>            | Estratégia: responde do cache; só vai à rede se não tiver                                                           |
| <b>CORS</b>                   | Mecanismo do navegador que restringe quais origens podem chamar uma API                                             |
| <b>Debounce</b>               | Atrasar uma ação até que ela pare de ser disparada por um intervalo                                                 |
| <b>DTO</b>                    | <i>Data Transfer Object</i> — o formato de saída da API, diferente do modelo do banco                               |
| <b>ESM</b>                    | <i>ECMAScript Modules</i> — o sistema de módulos padrão do JavaScript ( <code>import</code> / <code>export</code> ) |
| <b>Fail fast</b>              | Falhar imediatamente na inicialização em vez de degradar silenciosamente                                            |
| <b>HSTS</b>                   | Cabeçalho que obriga o navegador a usar HTTPS naquele domínio                                                       |
| <b>Injeção de dependência</b> | Receber a dependência por parâmetro em vez de importá-la                                                            |
| <b>JWT</b>                    | Token assinado que garante integridade (não sigilo) do seu conteúdo                                                 |
| <b>Maskable</b>               | Ícone com margem de segurança para sobreviver ao recorte do Android                                                 |
| <b>Migration</b>              | Arquivo versionado que descreve uma alteração no schema do banco                                                    |
| <b>Network-First</b>          | Estratégia: tenta a rede; cai no cache se ela falhar                                                                |
| <b>PWA</b>                    | <i>Progressive Web App</i> — aplicação web instalável, com Service Worker                                           |
| <b>Rainbow table</b>          | Tabela pré-computada de senha → hash, inutilizada pelo salt                                                         |
| <b>Rate limit</b>             | Limite de requisições por origem num intervalo                                                                      |
| <b>Repository</b>             | Camada que isola o acesso a dados do resto da aplicação                                                             |
| <b>Salt</b>                   | Valor aleatório embutido no hash, para senhas iguais gerarem hashes diferentes                                      |
| <b>Service Worker</b>         | Script que age como proxy entre a aplicação e a rede                                                                |
| <b>Soft delete</b>            | Exclusão lógica: marca a linha como excluída em vez de removê-la                                                    |
| <b>Stale-While-Revalidate</b> | Estratégia: responde do cache e atualiza em segundo plano                                                           |
| <b>Timing attack</b>          | Ataque que extrai informação do tempo de resposta, não do conteúdo                                                  |
| <b>Workspaces</b>             | Recurso do npm que liga pacotes locais por link simbólico                                                           |
| <b>Zod</b>                    | Biblioteca de validação em que o schema também gera o tipo TypeScript                                               |